📘 DMDA LAB MANUAL

Data Mining & Data Analytics

Weka 3.8.7 & R Programming

Weeks 1 – 12 | Complete Step-by-Step Procedures

📑 Table of Contents

Week	Topic	Algorithm / Technique	Tool
1	Data Preprocessing	Filters (Replace, Normalize, Discretize...)	Weka
2	Association Rules	Apriori	Weka
3	Association Rules	FP-Growth (Binary Conversion)	Weka
4	Classification	ID3 & J48 Decision Trees	Weka
5	Classification	Naive Bayes	Weka
6	Clustering	SimpleKMeans	Weka
7	Clustering	Hierarchical Clusterer	Weka
8	Basic Statistics	Mean, Median, Mode, Covariance, Regression	R
9	Regression Models	Linear, Multiple, Logistic, Poisson	R
10	Advanced Analytics	Time Series, Nonlinear Regression, Decision Tree	R
11	Distributions	Normal & Binomial Distribution	R
12	Hypothesis Testing	Chi-Square, t-Test, F-Test	R

🟦 WEEK 1 — Data Preprocessing

> **Objective:** Clean a raw dataset by handling missing values, converting types, normalizing, discretizing, and removing duplicates.

📄 Dataset — `student` (10 instances, 7 attributes)

```
@relation student
@attribute SID,numeric
@attribute age,numeric
@attribute gender,string
@attribute studyhrs,numeric
@attribute attendance,numeric
@attribute marks,numeric
@attribute result,string
@data
1,12,male,10,65,45,fail
2,23,female,3,76,80,pass
3,8,male,4,60,80,pass
4,?,female,3,96,30,fail
5,12,?,3,66,86,pass
6,13,female,12,96,100,pass
7,13,female,4,86,80,pass
8,23,female,?,76,80,pass
9,24,male,3,32,40,pass
10,29,male,4,76,20,?
```

> ⚠️ Notice the `?` marks — these are **missing values** in age, gender, studyhrs, and result.

🔧 Procedure

Step	Action	Navigation Path
1	**Load Dataset**	Weka Explorer → Preprocess → `Open file` → select `week1.txt`
2	**Replace Missing Values**	Filter → Choose → `unsupervised` → `attribute` → `ReplaceMissingValues` → **Apply**
3	**String to Nominal**	Filter → Choose → `unsupervised` → `attribute` → `StringToNominal` → click on filter name → set `attributeRange` to **3,7** → OK → **Apply**
4	**Set No Class**	At the bottom, change the Class dropdown to "**No class**"
5	**Replace Missing (again)**	Filter → Choose → `unsupervised` → `attribute` → `ReplaceMissingValues` → **Apply**
6	**Normalize**	Filter → Choose → `unsupervised` → `attribute` → `Normalize` → **Apply**
7	**Discretize**	Filter → Choose → `unsupervised` → `attribute` → `Discretize` → **Apply**
8	**Remove Duplicates**	Filter → Choose → `unsupervised` → **`instance`** → `RemoveDuplicates` → **Apply**

> 💡 **Key Notes:**

- `StringToNominal` range **3,7** targets `gender` (3rd) and `result` (7th).
- "No class" ensures filters apply equally to all attributes.
- Step 8 uses **`instance`** (not `attribute`) — it operates on rows.
- `Normalize` scales numeric values to [0, 1]. `Discretize` converts them into bins.

🟩 WEEK 2 — Association Rules using Apriori

> **Objective:** Generate association rules from the contact-lenses dataset using the Apriori algorithm.

📄 Dataset — `contact-lenses` (24 instances, 5 attributes, all nominal)

```
@relation contact-lenses

@attribute age {young, pre-presbyopic, presbyopic}
@attribute spectacle-prescrip {myope, hypermetrope}
@attribute astigmatism {no, yes}
@attribute tear-prod-rate {reduced, normal}
@attribute contact-lenses {soft, hard, none}

@data
young,myope,no,reduced,none
young,myope,no,normal,soft
young,myope,yes,reduced,none
young,myope,yes,normal,hard
young,hypermetrope,no,reduced,none
young,hypermetrope,no,normal,soft
young,hypermetrope,yes,reduced,none
young,hypermetrope,yes,normal,hard
pre-presbyopic,myope,no,reduced,none
pre-presbyopic,myope,no,normal,soft
pre-presbyopic,myope,yes,reduced,none
pre-presbyopic,myope,yes,normal,hard
pre-presbyopic,hypermetrope,no,reduced,none
pre-presbyopic,hypermetrope,no,normal,soft
pre-presbyopic,hypermetrope,yes,reduced,none
pre-presbyopic,hypermetrope,yes,normal,none
presbyopic,myope,no,reduced,none
presbyopic,myope,no,normal,none
presbyopic,myope,yes,reduced,none
presbyopic,myope,yes,normal,hard
presbyopic,hypermetrope,no,reduced,none
presbyopic,hypermetrope,no,normal,soft
```

```
presbyopic,hypermetrope,yes,reduced,none
presbyopic,hypermetrope,yes,normal,none
```

🔧 Procedure

Step	Action	Navigation Path
1	**Load Dataset**	Weka Explorer → Preprocess → `Open file` → select `week2.txt`
2	**Go to Associate**	Click the **Associate** tab at the top
3	**Choose Apriori**	Click `Choose` → select `Apriori`
4	**Run**	Click **Start**
5	**Adjust (if needed)**	Click on the `Apriori` text → change `lowerBoundMinSupport` and `minMetric` (confidence) → OK → **Start** again

> 💡 **Key Notes:**
>
> - All attributes are already nominal — **no preprocessing needed**.
> - Output rules look like: `tear-prod-rate=reduced ==> contact-lenses=none conf:(1)`
> - **Support** = how frequently items appear together. **Confidence** = how often the rule is true.

🟧 WEEK 3 — Association Rules using FP-Growth

> **Objective:** Convert all attributes to binary format and mine rules using FP-Growth.

📄 Dataset — `contact-lenses` (same as Week 2)

```
@relation contact-lenses

@attribute age {young, pre-presbyopic, presbyopic}
@attribute spectacle-prescrip {myope, hypermetrope}
@attribute astigmatism {no, yes}
@attribute tear-prod-rate {reduced, normal}
@attribute contact-lenses {soft, hard, none}

@data
young,myope,no,reduced,none
young,myope,no,normal,soft
young,myope,yes,reduced,none
```

```
young,myope,yes,normal,hard
young,hypermetrope,no,reduced,none
young,hypermetrope,no,normal,soft
young,hypermetrope,yes,reduced,none
young,hypermetrope,yes,normal,hard
pre-presbyopic,myope,no,reduced,none
pre-presbyopic,myope,no,normal,soft
pre-presbyopic,myope,yes,reduced,none
pre-presbyopic,myope,yes,normal,hard
pre-presbyopic,hypermetrope,no,reduced,none
pre-presbyopic,hypermetrope,no,normal,soft
pre-presbyopic,hypermetrope,yes,reduced,none
pre-presbyopic,hypermetrope,yes,normal,none
presbyopic,myope,no,reduced,none
presbyopic,myope,no,normal,none
presbyopic,myope,yes,reduced,none
presbyopic,myope,yes,normal,hard
presbyopic,hypermetrope,no,reduced,none
presbyopic,hypermetrope,no,normal,soft
presbyopic,hypermetrope,yes,reduced,none
presbyopic,hypermetrope,yes,normal,none
```

🔧 Procedure

Step	Action	Navigation Path
1	**Load Dataset**	Weka Explorer → Preprocess → `Open file` → select `week3.txt`
2	**Nominal to Binary**	Filter → Choose → `unsupervised` → `attribute` → `NominalToBinary` → **Apply**
3	**Set No Class**	Change Class dropdown to **"No class"**
4	**Nominal to Binary (2nd)**	Filter → Choose → `unsupervised` → `attribute` → `NominalToBinary` → **Apply**
5	**Numeric to Binary**	Filter → Choose → `unsupervised` → `attribute` → `NumericToBinary` → **Apply**
6	**Set Class back**	Change Class dropdown back to `contact-lenses`
7	**Numeric to Binary (2nd)**	Filter → Choose → `unsupervised` → `attribute` → `NumericToBinary` → **Apply**
8	**Go to Associate**	Click the **Associate** tab
9	**Choose FP-Growth & Run**	Click `Choose` → select `FPGrowth` → Click **Start**
10	**Adjust (if needed)**	Click `FPGrowth` text → change `lowerBoundMinSupport` & `minMetric` → OK → **Start** again

> 💡 **Key Notes:**
>
> - FP-Growth requires **binary (0/1) attributes** — that's why we convert twice.
> - After conversion, attributes look like `age=young_binarized=1_binarized` with 0/1 values.
> - FP-Growth is **faster** than Apriori for large datasets (avoids candidate generation).

🟥 WEEK 4 — Classification using Decision Trees (ID3 & J48)

> **Objective:** Build decision tree classifiers using ID3 and J48 algorithms, then visualize the tree.

📄 Dataset — `employee` (10 instances, 6 attributes, all nominal)

```
@relation employee
@attribute Age{Young, Middle, Senior}
@attribute Dept{IT, HR, Manager, Salesman}
@attribute Salary{Low, Medium, High}
@attribute Experience{Low, Medium, High}
@attribute Attendance{Poor, Avg, Good}
@attribute Performance{Poor, Avg, Good}
@data
Young, Manager, Low, Medium, Good, Avg
Middle, HR, High, Low, Poor, Avg
Senior, IT, Low, High, Avg, Poor
Middle, Salesman, Medium, Medium, Poor, Avg
Young, IT, High, Low, Good, Avg
Middle, HR, Low, High, Good, Good
Senior, Manager, High, Low, Poor, Avg
Young, Salesman, Low, High, Poor, Good
Senior, Manager, High, Medium, Good, Avg
Middle, IT, Medium, Low, Avg, Poor
```

> **Class attribute:** `Performance` (last attribute — Poor, Avg, Good)

🔧 Procedure

Step	Action	Navigation Path
1	**Load Dataset**	Weka Explorer → Preprocess → `Open file` → select `week4.txt`
2	**Go to Classify**	Click the **Classify** tab

Step	Action	Navigation Path
3	**Choose ID3 & Run**	Click `Choose` → `classifiers` → `trees` → `Id3` → Click **Start**
4	**Choose J48 & Run**	Click `Choose` → `classifiers` → `trees` → `J48` → Click **Start**
5	**Visualize Tree**	In **Result list** (left panel) → right-click on the J48 result → click **"Visualize tree"**

> 💡 **Key Notes:**
>
> - **ID3** works only with **nominal** attributes, cannot handle missing values.
> - **J48** (Java version of C4.5) handles both numeric and nominal, plus missing values.
> - The tree shows: **Oval nodes** = decision points, **Branches** = values, **Leaves** = predictions.

🟥 WEEK 5 — Classification using Naive Bayes

> **Objective:** Build a Naive Bayes classifier and evaluate its performance using cross-validation.

📄 Dataset — `employee9` (10 instances, 5 attributes, all nominal)

```
@relation employee9
@attribute Gender{Male,Female}
@attribute Age{Young, Middle, Senior}
@attribute Experience{Low, Medium, High}
@attribute Salary{Low, High}
@attribute Promotion{Yes, No}
@data
Male,Young,Low,High,No
Female,Senior,Medium,Low,Yes
Female,Middle,Low,High,Yes
Male,Middle,Medium,Low,Yes
Female,Young,High,High,No
Male,Middle,Low,High,No
Male,Senior,Low,Low,No
Female,Middle,Low,Low,Yes
Male,Middle,Low,High,No
Female,Young,Medium,High,No
```

> **Class attribute:** `Promotion` (last attribute — Yes / No)

🔧 Procedure

Step	Action	Navigation Path
1	**Load Dataset**	Weka Explorer → Preprocess → `Open file` → select `week5.txt`
2	**Go to Classify**	Click the **Classify** tab
3	**Choose Naive Bayes**	Click `Choose` → `classifiers` → `bayes` → `NaiveBayes`
4	**Set Test Option**	Select **Cross-validation** with **Folds: 10** (default)
5	**Run**	Click **Start**

📊 Reading the Output

Summary:

```
Correctly Classified Instances     6        60%
Incorrectly Classified Instances   4        40%
```

Confusion Matrix:

```
 a  b   ← classified as
 3  2 | a = Yes
 2  4 | b = No
```

> 💡 **How to read:** Rows = actual class, Columns = predicted class. `3 2 | a = Yes` → Out of actual "Yes", 3 predicted correctly, 2 predicted wrong.

🟪 WEEK 6 — Clustering using SimpleKMeans

> **Objective:** Cluster the Iris dataset using the K-Means algorithm.

📄 Dataset — `iris` (15 instances — 5 per species, 5 attributes)

```
@RELATION iris

@ATTRIBUTE sepallength REAL
@ATTRIBUTE sepalwidth REAL
@ATTRIBUTE petallength REAL
@ATTRIBUTE petalwidth REAL
@ATTRIBUTE class {Iris-setosa,Iris-versicolor,Iris-virginica}
```

```
@DATA
5.1,3.5,1.4,0.2,Iris-setosa
4.9,3.0,1.4,0.2,Iris-setosa
4.7,3.2,1.3,0.2,Iris-setosa
4.6,3.1,1.5,0.2,Iris-setosa
5.0,3.6,1.4,0.2,Iris-setosa
7.0,3.2,4.7,1.4,Iris-versicolor
6.4,3.2,4.5,1.5,Iris-versicolor
6.9,3.1,4.9,1.5,Iris-versicolor
5.5,2.3,4.0,1.3,Iris-versicolor
6.5,2.8,4.6,1.5,Iris-versicolor
6.3,3.3,6.0,2.5,Iris-virginica
5.8,2.7,5.1,1.9,Iris-virginica
7.1,3.0,5.9,2.1,Iris-virginica
6.3,2.9,5.6,1.8,Iris-virginica
6.5,3.0,5.8,2.2,Iris-virginica
```

🔧 Procedure

Step	Action	Navigation Path
1	**Load Dataset**	Weka Explorer → Preprocess → `Open file` → select `week6.txt`
2	**Go to Cluster**	Click the **Cluster** tab
3	**Choose SimpleKMeans**	Click `Choose` → select `SimpleKMeans`
4	**Set Cluster Mode**	Select **"Use training set"**
5	**Run**	Click **Start**
6	**Change Clusters (if needed)**	Click on `SimpleKMeans` text → change `numClusters` to **3** (3 species) → OK → **Start** again

> 💡 **Key Notes:**
>
> - Default `numClusters` = 2. Change to **3** for Iris (3 species).
> - Output shows **cluster centroids** (mean values) and instance counts per cluster.

🟫 WEEK 7 — Clustering using Hierarchical Clusterer

> **Objective:** Perform hierarchical clustering and visualize the dendrogram.

📄 Dataset — `student` (10 instances, 5 attributes, all nominal)

```
@relation student
@attribute Age {<=25,30-40,>40}
@attribute Income {Low,Medium,High}
@attribute Student {Yes,No}
@attribute CreditcardRating {Poor,Fair,Excellent}
@attribute Computer {Yes,No}
@data
<=25,Medium,Yes,Excellent,Yes
>40,Medium,No,Fair,Yes
30-40,Low,Yes,Fair,No
>40,Low,Yes,Excellent,No
<=25,High,Yes,Fair,No
>40,Low,Yes,Fair,No
30-40,Medium,Yes,Poor,Yes
<=25,Low,Yes,Fair,No
30-40,High,No,Poor,Yes
>40,Low,Yes,Excellent,No
```

🔧 Procedure

Step	Action	Navigation Path
1	**Load Dataset**	Weka Explorer → Preprocess → `Open file` → select `week7.txt`
2	**Go to Cluster**	Click the **Cluster** tab
3	**Choose Hierarchical Clusterer**	Click `Choose` → select `HierarchicalClusterer`
4	**Enable Visualization**	✅ Check **"Store clusters for visualization"** (left panel)
5	**Run**	Click **Start**
6	**Visualize Dendrogram**	In **Result list** (left) → right-click on result → click **"Visualize tree"**

> ⚠️ You **must** check "Store clusters for visualization" **before** clicking Start, otherwise "Visualize tree" won't appear!

📗 R PROGRAMMING — Weeks 8 to 12

Open RStudio → Type code in Console or R Script → Run with Ctrl+Enter

🟢 WEEK 8 — Basic Statistics in R

> **Objective:** Compute Mean, Median, Mode, Covariance and perform simple Regression.

📌 8.1 — Mean

```
x = c(10, 20, 30, 40, 50)
result = mean(x)
print(result)
```

> **Output:** `30` — The average of all values.

📌 8.2 — Median

```
x = c(10, 20, -30, 40, -50, -10, -20)
result = median(x)
print(result)
```

> **Output:** `-10` — The middle value when data is sorted.

📌 8.3 — Mode

> R has **no built-in mode function**, so we create one:

```
getmode = function(v)
{
  uniqv = unique(v)
  uniqv[which.max(tabulate(match(v, uniqv)))]
}
v = c(10, 20, 10, 30, 20, 10, 40)
result = getmode(v)
print(result)
```

> **Output:** `10` — The most frequently occurring value.

📌 8.4 — Covariance

```
x = c(10, 20, 30, 40)
y = c(50, 60, 70, 80)
result = cov(x, y)
print(result)
boxplot(x, y)
```

> **Output:** `166.6667` — Positive covariance means both variables increase together.

📌 8.5 — Regression

```
x = c(25, 35, 45, 60)
y = c(2, 11, 10, 25)
result = lm(x ~ y)
print(result)
boxplot(x ~ y)
```

> `lm()` fits a **linear model**. Output shows **intercept** and **slope** coefficients.

🔧 Procedure

Step	Action
1	Open **RStudio**
2	Type each code block in the **Console** or create a new **R Script**
3	Run each block using **Ctrl+Enter**
4	Observe the printed output and generated plots

🔵 WEEK 9 — Regression Models in R

> **Objective:** Build four types of regression models and visualize the results.

📌 9.1 — Linear Regression

```
stdhrs <- c(1, 2, 3, 4, 5)
marks <- c(40, 50, 60, 70, 80)
```

```
student <- data.frame(stdhrs, marks)
print(student)
model = lm(marks ~ stdhrs, data = student)
plot(stdhrs, marks, main = "linear regression",
     xlab = "stdhrs", ylab = "marks", pch = 19, col = "blue")
abline(model, col = "purple", lwd = 2)
```

> Fits a straight line — as study hours ↑, marks ↑. `abline()` draws the regression line.

📌 9.2 — Multiple Regression

```
experience <- c(1, 2, 3, 4, 5, 6)
education <- c(12, 12, 15, 15, 16, 18)
salary <- c(25000, 28000, 35000, 42000, 50000, 60000)
employee <- data.frame(experience, education, salary)
print(employee)
model <- lm(salary ~ experience + education, data = employee)
plot(employee$experience, employee$salary,
     main = "Experience vs Salary",
     xlab = "Experience(Years)", ylab = "Salary",
     pch = 19, col = "blue")
abline(lm(salary ~ experience))
```

> Predicts salary using **two** variables: `salary ~ experience + education` .

📌 9.3 — Logistic Regression

```
students <- data.frame(
  studyhrs = c(1, 2, 3, 4, 5, 6, 7, 8),
  pass     = c(0, 0, 0, 1, 1, 1, 1, 1)
)
model <- glm(pass ~ studyhrs, data = students, family = binomial)
summary(model)
students$Probability <- predict(model, type = "response")
students$Prediction <- ifelse(students$Probability >= 0.5, 1, 0)
students
plot(students$studyhrs, students$pass, pch = 19,
     xlab = "Study Hours", ylab = "Probability of Passing",
     main = "Logistic Regression")
```

> Predicts **binary outcome** (pass=1/fail=0). `glm()` with `family = binomial` fits a logistic curve.

📌 9.4 — Poisson Regression

```
hours <- c(1, 2, 3, 4, 5, 6, 7, 8)
questions <- c(2, 3, 4, 6, 8, 10, 13, 16)
data <- data.frame(hours, questions)
print(data)
model <- glm(questions ~ hours, family = poisson, data = data)
summary(model)
predicted <- predict(model, type = "response")
print(predicted)
plot(data$hours, data$questions, pch = 19,
     xlab = "Study Hours", ylab = "Questions Answered",
     main = "Poisson Regression")
lines(data$hours, predicted, lwd = 2)
```

> Predicts **count data**. `family = poisson` is used when output is a count (0, 1, 2...).

🔧 Procedure

Step	Action
1	Open **RStudio**
2	Type each regression code block in the **Console** or **R Script**
3	Run using **Ctrl+Enter**
4	Observe the **summary output** (coefficients, p-values) and **plots**

🔴 WEEK 10 — Time Series, Nonlinear Regression & Decision Tree in R

> **Objective:** Perform time series analysis, nonlinear regression, and build a decision tree.

📌 10.1 — Time Series Analysis

```
sales <- c(100, 200, 300, 400, 500, 600,
           700, 800, 900, 1000, 1100, 1200)
sales_ts <- ts(sales, start = c(2025, 1), frequency = 12)
print(sales_ts)
plot(sales_ts, type = "o",
```

```
     main = "time series analysis",
     xlab = "months", ylab = "sales", lwd = 3)
```

> `ts()` creates a time series. `start = c(2025, 1)` = January 2025. `frequency = 12` = monthly.

📌 10.2 — Nonlinear Least Squares (NLS)

```
year = c(1, 2, 3, 4, 5, 6)
population = c(100, 110, 125, 140, 160, 185)
model = nls(population ~ a * exp(b * year),
            start = list(a = 90, b = 0.1))
print(model)
predicted = predict(model)
print(predicted)
plot(year, population, pch = 19,
     main = "population growth-non linear regression",
     xlab = "years", ylab = "population(thousands)")
lines(year, predicted, lwd = 2)
new_year = data.frame(year = 8)
predicted_8 = predict(model, newdata = new_year)
```

> Fits an **exponential curve**: `population = a × e^(b × year)` . Also predicts for year 8.

📌 10.3 — Decision Tree (using rpart)

```
if (!require(rpart))
  install.packages("rpart")
if (!require(rpart.plot))
  install.packages("rpart.plot")
library(rpart)
library(rpart.plot)
data(mtcars)
reg_tree = rpart(mpg ~ hp + wt, method = "anova", data = mtcars)
rpart.plot(reg_tree, type = 2, digits = 3,
           fallen.leaves = TRUE,
           main = "R Decision Tree")
```

> Builds a **regression decision tree** predicting `mpg` from `hp` and `wt` using the built-in `mtcars` dataset.

🔧 Procedure

Step	Action
1	Open **RStudio**
2	For Decision Tree: install `rpart` and `rpart.plot` packages if not already installed
3	Type and run each code block using **Ctrl+Enter**
4	Observe the **time series plot**, **exponential curve**, and **decision tree diagram**

🟡 WEEK 11 — Probability Distributions in R (Normal & Binomial)

> **Objective:** Generate and visualize Normal and Binomial distributions using R's built-in functions.
>
> 💡 **Tip:** `d` = density, `p` = cumulative probability, `q` = quantile (inverse), `r` = random.

📌 11.1 — Normal Distribution

i. `dnorm()` — Probability Density Function

```
x <- seq(-10, 10, by = 0.1)
y <- dnorm(x, mean = 2.5, sd = 0.5)
model = lm(y ~ x)
plot(x, y, main = "Normal Distribution",
     xlab = "X", ylab = "Density", type = "l")
abline(model, col = "blue", lwd = 2)
```

> Returns the **height of the bell curve** at each point x. The blue line shows the linear trend.

ii. `pnorm()` — Cumulative Distribution Function

```
x <- seq(-10, 10, by = 0.1)
y <- pnorm(x, mean = 2.5, sd = 0.5)
plot(x, y, main = "Cumulative Normal Distribution",
     xlab = "X", ylab = "Cumulative Probability", type = "l")
```

> Returns **P(X ≤ x)** — the probability that a value is less than or equal to x.

iii. `qnorm()` — Quantile Function (Inverse CDF)

```
x <- seq(0, 1, by = 0.02)
y <- qnorm(x, mean = 2, sd = 1)
plot(x, y, main = "Quantile Normal Distribution",
     xlab = "X", ylab = "Quantile Distribution", type = "l")
```

> Given a probability, returns the **x value** at that probability (inverse of `pnorm`).

iv. `rnorm()` — Random Normal Values

```
y <- rnorm(50)
hist(y, main = "Random Normal Distribution",
     xlab = "Values", col = "lightblue")
```

> Generates **50 random values** from a standard normal distribution and plots a histogram.

📌 11.2 — Binomial Distribution

i. `dbinom()` — Probability Mass Function

```
x <- seq(0, 50, by = 1)
y <- dbinom(x, 50, 0.5)
plot(x, y, main = "Binomial Distribution",
     xlab = "Number of Successes",
     ylab = "Probability", type = "h")
```

> Returns the probability of getting **exactly x successes** in 50 trials with p=0.5.

ii. `pbinom()` — Cumulative Distribution Function

```
x <- 0:51
y <- pbinom(x, 51, 0.5)
plot(x, y, main = "Cumulative Binomial Distribution",
     xlab = "Number of Successes",
     ylab = "Cumulative Probability", type = "s")
```

> Returns **P(X ≤ x)** — probability of getting x or fewer successes.

iii. `qbinom()` — Quantile Function

```
x = seq(0, 1, by = 0.02)
y = qbinom(x, 2.0, 0.5)
plot(x, y, main = "Quantile Binomial Distribution",
```

```
     xlab = "Number of Success",
     ylab = "Quantile", type = "s")
```

> Returns the number of successes at each probability quantile.

iv. `rbinom()` — Random Binomial Values

```
y = rbinom(50, 10, 0.5)
hist(y, main = "Random Binomial Distribution",
     xlab = "values", col = "blue")
```

> Generates **50 random values** from a binomial distribution (10 trials, p=0.5).

🔧 Procedure

Step	Action
1	Open **RStudio**
2	Type each distribution code block in the **Console** or **R Script**
3	Run using **Ctrl+Enter**
4	Observe the **density plots**, **cumulative curves**, **histograms**, and printed values

🔴 WEEK 12 — Statistical Hypothesis Testing in R

> **Objective:** Perform Chi-Square, t-Test, and F-Test to determine statistical significance.

📌 12.1 — Chi-Square Test

```
library("MASS")
print(str(Cars93))
car = data.frame(Cars93$AirBags, Cars93$Type)
car = table(Cars93$AirBags, Cars93$Type)
print(car)
smoothScatter(car)
print(chisq.test(car))
```

> Tests whether **AirBags type** and **Car Type** are **independent**. Uses built-in `Cars93` dataset. If **p-value < 0.05** → they are **related** (not independent).

📌 12.2 — One-Sample t-Test

```
x = c(0.593, 0.142, 0.329, 0.691, 0.231,
      0.793, 0.392, 0.418)
result = t.test(x, alternative = "greater", mu = 0.3)
print(result)
hist(x, probability = TRUE,
     main = "One-Sample t-Test",
     xlab = "Sample Values", ylab = "Density",
     col = "lightblue", border = "Green")
```

> Tests whether the **mean of x > 0.3**. `alternative = "greater"` = one-sided test. If **p-value < 0.05** → mean is significantly greater than 0.3.

📌 12.3 — F-Test (Variance Test)

```
x = c(66, 67, 75, 76, 82, 84, 88, 90, 92)
y = c(64, 66, 74, 78, 82, 85, 87, 92, 93, 95, 97)
result = var.test(x, y)
print(result)
boxplot(x, y,
        names = c("Group X", "Group Y"),
        main = "Comparison of Variances",
        xlab = "Groups", ylab = "Values",
        col = c("lightblue", "lightgreen"))
```

> Tests whether **variances** of two groups are **equal**. If **p-value < 0.05** → they are significantly different.

🔧 Procedure

Step	Action
1	Open **RStudio**
2	For Chi-Square: load `MASS` library (pre-installed in R)
3	Type and run each test code block using **Ctrl+Enter**
4	Observe the **test statistics**, **p-values**, and **plots**

Step	Action
5	Interpret: if **p-value < 0.05**, the result is **statistically significant**

📋 Quick Reference — All Weeks

Week	Tool	Algorithm	Dataset	What You Do
1	Weka	7 Filters	student (10)	Clean & transform raw data
2	Weka	Apriori	contact-lenses (24)	Mine association rules
3	Weka	FP-Growth	contact-lenses (24)	Binary convert → mine rules
4	Weka	ID3 & J48	employee (10)	Build & visualize decision trees
5	Weka	NaiveBayes	employee9 (10)	Probabilistic classification
6	Weka	SimpleKMeans	iris (15)	Partition data into K clusters
7	Weka	HierarchicalClusterer	student (10)	Hierarchical clustering + dendrogram
8	R	Mean, Median, Mode, Cov	—	Basic statistics
9	R	Linear, Multiple, Logistic, Poisson	—	Regression models
10	R	Time Series, NLS, Decision Tree	—	Advanced analytics
11	R	Normal & Binomial Distribution	—	Probability distributions
12	R	Chi-Square, t-Test, F-Test	—	Hypothesis testing